CST1500 Re-sit Coursework

Deadline for submission: Monday 14th April 2025 18:00.

NOTE: Due to expected high use, there will be a 2-hour window where submissions will be accepted and will not incur a late penalty. Submissions received after 20:00 will be treated as late and will lose 10% of their final mark (to a limit of no lower than 40%, providing the final mark before penalties is above 40%). Submissions received more than 24 hours late will receive zero marks. DO NOT TREAT THIS AS A 20:00 DEADLINE – I advise starting your submission as early as possible before the 18:00 deadline.

Please read all the assignment specification carefully first, before asking your tutor any questions.

General Information

The re-sit coursework for CST1500 is a portfolio of work to be completed by the specified deadline. The portfolio will comprise six individual pieces of work: five smaller tasks and one more significant task. The deliverable for each task is specified on pages 4 - 6. All six tasks (or all attempted/completed tasks) must be submitted to the dedicated assignment link in the Re-Sit Coursework folder on the module page by the specified deadline.

Your work will not be accepted as an email attachment.

Therefore, you are strongly advised to allow plenty of time to upload your work prior to the deadline.

NOTE: This is an **INDIVIDUAL** Coursework and should **NOT** be completed in pairs or in groups. Please refer to Section 6 of the module handbook with regards to integrity and academic misconduct.

What to Submit

A zipped folder containing all of the files and folders as specified in each task RCWTASK1 – RCWTASK6.

IMPORTANT: To enable consistency and ease of marking the folder should be named **CST1500RCW** and the filename of each task should be in the following format:

RCWTASK1
RCWTASK2
RCWTASK3 etc.

For example: RCWTASK1.c, RCWTASK4.bat, etc.

IMPORTANT NOTE: Some tasks require you to create additional files and folders. These files and folders should also be present in your submission.

Deadline

Monday 14th April 2025 18:00.

Where to submit

Submit your zipped folder to the dedicated submission link in the Re-Sit Coursework folder on the course page.

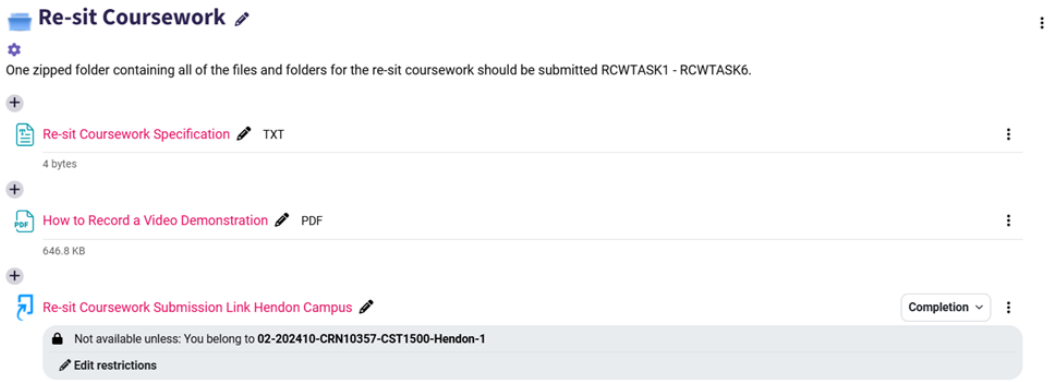

Fig. 1: Re-Sit Coursework Submission link on the CST1500 course page.

Demonstration of each Task

Each Task must be demonstrated in a video demonstration.

During the demonstration you should run your program/script and show the output. If you have not completed a Task, you can show what you have done.

It is important to show that you understand the code and can say something useful and interesting about the program to show your understanding of the code.

You should make a separate video for each Task. Each video for RCWTASK1 – RCWTASK5 should be 1 – 2 minutes – 2 minutes maximum.

The video for RCWTASK6 should be 5 minutes maximum.

At the beginning of each video please state your name and student ID.

Name the video file the same as the Task to which it pertains i.e. a video demo for RCWTASK4 would be named RCWTASK4.mp4.

Details of how to record a video demonstration can be found on the course page in the Re-sit Coursework folder – How to Record a Video Demonstration.

IMPORTANT

If you do not demonstrate your work via a submitted video, marks will be deducted from each Task not demonstrated – see the below marking criteria.

Marking

This piece of coursework will be capped at **40%** which is the equivalent of **16** on the 20-point scale.

YOU MUST ACHIEVE AT LEAST 40% TO PASS THE COURSEWORK AND THE MODULE.

Each of the 5 smaller Tasks (RCWTASK1 – RCWTASK5) carry a weighting of **10%** and RCWTASK6 carries a weighting of **50%**. Each **10%** for each Task **RCWTASK1 – RCWTASK5** is distributed and applied using the following relative marking criteria:

Criterion	Marks
Correctness of output	4
Code/script formatting and sensible use of comments in code	2
Following instructions	2
Demo of work – in class or video	2

Half Marks will be awarded where appropriate.

The **50%** for **RCWTASK6** will be marked using the following criteria:

Item	1-4	5-8	9-12	13-16	Fail (17-20)
FCFS Code	High quality code exhibiting deep knowledge and understanding of Python and FCFS (comprehensive comments in code)	Code shows good structure, good understanding of Python and FCFS with good personal initiative (good comments in code)	Code shows some personal initiative and good understanding of basic Python and FCFS concepts (some comments in code)	Code works, mostly based on Python and FCFS examples with little personal contribution (a few comments in code)	Code does not work, and shows little understanding of Python and FCFS algorithm (no comments in code)
Demonstration	Very clear and organised demonstration of code (practise evident)	Clear and organised demonstration of code	Demonstration is clear but not very organised	Basic explanation of code	No clear explanation, lack of understanding. No video demo

All marks are provisional and subject to adjustment during moderation of the overall module assessment which takes place at the end of the year with external examiner oversight.

Use of AI

AI is a powerful tool that can increase the productivity of programmers. You are welcome to use AI tools like GitHub Copilot, ChatGPT and CodeWhisperer to help you to write code for your coursework. You are also welcome to use text generation tools like ChatGPT and Microsoft Copilot to help write your reports.

AI tools can help you to program and write more efficiently, but they are not going to complete the coursework for you, they frequently make mistakes, and they are unlikely to follow the technology requirements in the coursework descriptions. If you use these tools to generate the code and report for you without understanding the technologies or coursework requirements, then you will get a low mark for this module.

I strongly recommend that you spend time understanding how the programming languages (C, Bash, Batch, Python etc.) work and write code with these technologies without the help of AI. Then you will be in a much better position to use AI tools to help you to generate drafts of pieces of code for your project. Your understanding of the technologies will enable you to change this code to match the project requirements, integrate it, and fix bugs.

Detailed specification of each Task

RCWTASK1 – A C program with user input

Write a **C program** that take **TWO** numbers entered by the user. The first number should be less than 10, the second number should be more than 20. Some operations should be performed on the numbers (+, -, /, and *).

The program should also take the name of the user and display it in every result e.g.

```
Student_Name has entered X and Y. X + Y equals Z.
Student_Name has entered X and Y. X * Y equals Z.
```

Where `Student_Name`, `X` and `Y` are the details entered by the user and `Z` is the result of the calculations.

Include some relevant comments in your code.

RCWTASK2 – A C program password entry system

Write a **C programme** that does the following:

1. Prompt the user for a whole number `X`.
2. Pass `X` to a method/function called “printName”.
3. The “printName” method/function should prompt the user for their name (`userName`) and then print that name `X` times.
4. Following this, the program should invoke a separate method/function called “passwordEntry”. The “passwordEntry” method/function should prompt the user, by their provided name, to enter a password. The password will be checked against a sentinel value (CST1500).
5. The user must enter the correct password within 3 attempts or they will be locked out of the program.
6. The program should adequately advise the user of all instructions and warnings throughout.
7. The final output should look like the following:

   Success
   Well done `userName`, you got the password right in `X` attempts!

   Failure
   Bad luck `userName`, you took `X` attempts, you are locked out!

Include some relevant comments in your code.

RCWTASK3 – A C program with user input and control structure

Write a **C program** that takes an exam score which is represented by an integer and provide a string result which is the classification. Include the score in the output.

The classification is as follows:

0 - 39 = FAIL
40 – 49 = THIRD
50 - 59 = LOWER SECOND
60 – 69 = UPPER SECOND
70 – 100 = FIRST

Example Input: 55

Example Output: Your exam score is 55 which is a LOWER SECOND.

The program should continue to prompt for grade input until the user enters -1.

Include some relevant comments in your code.

RCWTASK4 – A Batch Script which creates files and folders

Write a **Batch Script** that will create 3 text files and 3 folders. Each folder should contain one of the created text files. One text file should contain your first name. One text file should contain your student ID. One text file should contain the module number. All 6 items should be in a folder named “The_Bag”. Once completed, the following message should be displayed:

“The 6 items are in The Bag!”.

Include some relevant comments in the script.

RCWTASK5 – A Bash Script exercise

Write a **Bash Script** that will take an existing directory as an input, the script must create three folders within that path called “a”, “b” and “c” and perform the following tasks:

Task 1: Store the name of your favourite movie in a text file called “myFavMovie.txt” in folder “a”.

Task 2: Store your student ID in a text file called “userID.txt” in folder “b”.

Task 3: Move folders “a” and “b” with their contents to folder “c”.

Include some relevant comments in the script.

RCWTASK6 – A CPU Scheduling Algorithm in Python

You are required to write a program using the Python programming language. The program should demonstrate the First Come First Served (FCFS) CPU scheduling algorithm.

We will be looking for well-structured code which uses Python programming conventions, sensible naming conventions and contains comprehensive code comments.

We will also be looking for correct output from the code, or in the case of partially working code, the expected output should be displayed in your report in the form of a table.

The algorithm should correctly implement the functionality according to its specification. You can find brief details of each algorithm in the lecture video Lecture 13 CPU Scheduling on the course page. **Further investigation is recommended**.

First Come First Served (FCFS) CPU Algorithm

Your program should output:

Process Number
Burst Time
Waiting Time
Turn Around Time
Average Waiting Time
Average Turn Around Time

AT A MINIMUM your program should provide output for the following input:

Process Number	Burst Time
1	6
2	9
3	13

User input is recommended to improve the potential for higher marks.